ESLtopia — Master Test Plan

Worksheet Generator & Classroom Management Platform · Test Plan v1.1 · 2026-07-03

Field	Value
Application	ESLtopia (English worksheet generator + classroom management)
Production URL	`https://esltopia.vercel.app`
Repository	`github.com/JovovicMiljan89/ESLtopia`
Document version	1.1
Status	Active
Related documents	Test Cases Specification, Test Scenario Coverage Checklist

> **v1.1 revision:** `EnglishGenerator.jsx` was split from a single ~4,750-line file into 8 focused modules (§2 updated below); a code review pass fixed one critical data-loss-warning gap, two silent-error-handling bugs, a stale-closure bug, a biased randomization function, and deduplicated a test helper (see the Test Case Specification's revision note for details). No test cases were added or removed — the 126-case suite is unchanged in count.

1. Introduction & Purpose

This document defines the testing strategy for ESLtopia, a React/Vite single-page application backed by Supabase (Postgres, Auth, Edge Functions) and deployed to Vercel. It covers what is tested, how, where, with what data, and what "done" means for a release. It is the reference companion to the **Test Cases Specification** (detailed, step-by-step cases mapped to the automated suite) and the **Test Scenario Coverage Checklist** (the full list of scenarios that should be covered, including gaps not yet automated).

2. Application Under Test — Overview

ESLtopia lets teachers (and schools managing teachers) generate printable English-language worksheets for grades 1–6 across 24 topics and five exercise types, preview/print/download them as a formatted A4 PDF, and manage classes, students, and per-student attendance/grade/payment records. Three roles exist: **teacher**, **school**, and **superadmin**, enforced end-to-end via Postgres Row-Level Security (RLS).

Layer	Technology
Frontend	React 18, Vite 6, split into focused modules under `src/` : `EnglishGenerator.jsx` (main orchestrator, ~950 lines), `AuthScreen.jsx` , `AdminPanel.jsx` , `EmployeesPanel.jsx` , `LandingPage.jsx` , `PdfPreviewModal.jsx` , `WorksheetTasks.jsx` , plus data/style modules (`worksheetContent.js` , `profileHelpers.js` , `styles.js`)
Backend	Supabase (Postgres + RLS, GoTrue Auth, Edge Functions on Deno)
Email	Gmail SMTP (confirmation / invite / password-reset mail)
Hosting	Vercel (manual `vercel --prod` deploys; not auto-triggered by git push)
Test framework	Playwright + TypeScript, Chromium only

3. Objectives

- Verify that every user-facing flow (auth, worksheet generation, PDF preview, classes, records, admin/school management) behaves correctly for each role.
- Verify that Postgres RLS policies and DB triggers correctly enforce data isolation and prevent privilege escalation.
- Catch regressions before they reach real users of a live, low-traffic production app with no staging environment.
- Keep the suite fast and deterministic enough to run before every deploy.

4. Scope

4.1 In scope

- Authentication: signup, login, logout, forgot/reset password, session persistence, account status gating (pending/inactive).
- Authorization: teacher / school / superadmin role boundaries, RLS isolation, privilege-escalation prevention, JWT claim correctness.
- Superadmin Admin Panel: user visibility, role changes, profile deletion.
- School → Teachers management (Employees panel + edge functions), currently gated behind `FEATURE_SCHOOL_TEACHERS`.
- Classes: CRUD, student roster management, RLS isolation, cascade delete of records.
- Records: attendance / grades / payment tracking UI, per-student profile modal, notes, debounced Supabase sync.
- Worksheet Generator: topic/grade selection, task-type switching, worksheet generation logic.
- PDF Preview Modal: open/close (button + Escape), answer-key toggle, regenerate ("New set"), print/download triggers.

4.2 Out of scope (for this suite)

- Cross-browser testing (Firefox / WebKit / Safari / mobile browsers) — Chromium only today.
- Visual regression / pixel-diff testing of the worksheet PDF layout.
- Load, stress, and performance testing (no defined SLA; ~10-user portfolio app).
- Accessibility audit beyond incidental keyboard support (Escape-to-close).
- Actual email content/deliverability verification (tests use the admin API instead of reading an inbox).
- Native mobile apps (none exist).
- Unit/component-level testing — the project deliberately uses a single E2E layer (see §6).

See the Test Scenario Coverage Checklist for the authoritative, itemized list of what is and isn't automated within the in-scope areas above.

5. Test Levels & Types

Type	Description	Example
UI / E2E	Drives the real browser against production, asserting on rendered DOM state.	Toggling the PDF answer-key switch; marking a student present in the Records tab.
API / REST	Calls Supabase Auth/REST endpoints directly with `request`, bypassing the UI, to assert on RLS and trigger behavior precisely.	PATCH a profile's `role` column and confirm the DB trigger rejects it.
Security / RLS	Confirms Postgres policies isolate data between roles/owners and block privilege escalation.	Teacher B cannot read Teacher A's class.
Regression	The full suite, run as a whole before any production deploy.	All ~136 tests across 12 spec files.

6. Test Strategy

The suite is intentionally a single E2E layer rather than split unit/integration/E2E layers: the app is a single component file with no isolated business-logic modules to unit-test in isolation, and the primary risk surface is Supabase RLS/trigger behavior plus real user flows — both are best verified against the real stack. Where a UI click would just be a thin wrapper over a REST call already covered elsewhere (e.g. class/record CRUD), tests call the REST API directly for precision and speed; where the click handler itself is the thing under test and has no other coverage (e.g. Records tab toggles, PDF modal interactions), tests drive the real UI.

> Tests run against **production** (`https://esltopia.vercel.app`), not a staging environment — there isn't one. This is a deliberate tradeoff for a low-traffic portfolio app; see §11 (Risks) for how this is made safe.

7. Test Environment

Item	Detail
Target	Production — `https://esltopia.vercel.app` (override with `BASE_URL` env var)
Backend project	Supabase `jogbbmssrdlujzlxwiji`

Browser	Chromium (headless), via Playwright
Parallelism	`fullyParallel: true` locally; `workers: 1` on CI
Secrets	`.env.test.local` (gitignored): `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`
Feature flags	`FEATURE_SCHOOL_TEACHERS` — gates the school→teachers edge-function tests until that feature is deployed

8. Test Data Management

- All test accounts use the `@example.test` domain via a shared `uniqueEmail(prefix)` helper, so they're unambiguously identifiable as fixtures.
- Fixture users are created pre-confirmed via the Supabase Admin API (`createConfirmedUser`) rather than the real signup+email-confirmation flow, for speed and determinism.
- A global teardown (`tests/global-teardown.ts`) purges every `@example.test` user after each run; cascading deletes remove their classes and records automatically.
- Tests that mutate shared state (e.g. Records tab) create their own isolated fixtures (a fresh class per test) rather than sharing one, because the suite runs fully parallel and shared mutable fixtures would race.

9. Entry Criteria

- Feature branch merged/pushed to `main` and, if the change is user-facing, deployed to production (`vercel --prod --yes` — not automatic on push).
- `.env.test.local` present with valid Supabase credentials.
- No known Supabase outage or maintenance window in progress.

10. Exit Criteria

- All tests pass, or fail only with a previously-known, documented flake (currently: two auth specs occasionally need their configured retry).
- No new browser console errors observed during manual/agent-driven verification of the changed feature.
- New features ship with either automated coverage or an explicit, reviewed decision to defer it (recorded in the Test Scenario Coverage Checklist as a gap).

11. Risks & Mitigations

Risk	Mitigation
Tests run against production — a bug in a test could affect real data.	All fixture data is confined to the `@example.test` domain and purged by global teardown; RLS means fixture users can never touch real users' rows.
Debounced writes (e.g. Records autosave, 800ms) have no visible "saved" indicator.	Persistence-sensitive tests explicitly wait out the debounce window, then reload and re-read from the server rather than trusting client state.
Fully parallel execution can race tests that share mutable fixtures.	Each test that mutates shared-looking state (classes, records) creates its own isolated fixture instead of reusing one across a describe block.
Frontend deploys are not automatic on git push.	Documented explicitly; a production deploy step (`vercel --prod --yes`) with bundle-hash verification is a required part of shipping a UI change before UI tests are re-run against it.
Email-dependent specs (registration, school-teachers) can flake on SMTP timing.	Those describe blocks set `retries: 2`.
A feature (school-teachers) is code-complete but not yet deployed.	Its edge-function tests are marked `test.skip` with an explicit reason, not silently omitted.

12. Roles & Responsibilities

Role	Responsibility
Developer / test author	Writes and maintains specs alongside feature code; runs the full suite before a production deploy.
Reviewer (future collaborators)	Reviews new specs for adherence to the conventions in §13 before merge.

13. Conventions & Tooling

- Shared helpers: `tests/helpers/cleanup.ts` (Supabase admin operations), `tests/helpers/ui.ts` (`loginToApp`), and `tests/helpers/edgeFunctions.ts` (`invokeEdgeFunction` — extracted from two byte-identical copies previously duplicated in `school-teachers.spec.ts` and `superadmin.spec.ts`).
- One `test.describe` per logical feature area; each describe block sets up its own fixtures.
- Non-auth feature specs live under `tests/app/`; auth-related specs live under `tests/auth/`.
- `npx playwright test --list` is run before trusting any new spec parses correctly.

14. Deliverables

- This Test Plan.
- Test Cases Specification — every automated test, with steps and expected results.
- Test Scenario Coverage Checklist — the full scenario inventory, automated and not.
- The automated spec files themselves, under `tests/`.
- The Playwright HTML report, generated per run (`npx playwright show-report`).